

From PhyloGFN to PhyloGFN-Net

A self-lecture on turning the PhyloGFN tree sampler
into a sampler of level-1 phylogenetic networks

(paper: Zhou et al., ICLR 2024 — code: <https://github.com/zmy1116/phylogfn>)

Prepared for the doctoral candidate, PDEIC / INESC-ID

September 2026

Abstract

This document teaches, from zero, everything needed to understand, present and defend a concrete transformation of the PhyloGFN code base: the original program builds *phylogenetic trees* by merging pairs of lineages; the modified program (*PhyloGFN-Net*) builds *level-1 phylogenetic networks* by adding one new action, RETICULATE, three legality masks, a two-version representation of partial likelihoods, a reticulation prior, and the corresponding policy heads, backward policy and marginal-likelihood estimator. Every modification is listed with the code that implements it and the reason it is needed. Every claim about the new code is backed by a test that you can run: the new Markov decision process reaches *exactly* the known class of level-1 networks (603 networks for four taxa, 11 460 for five), it has no dead ends, its fast likelihood agrees with an explicit enumeration of the 2^R displayed trees to 10^{-11} , and with the reticulation budget set to zero it reproduces PhyloGFN.

Contents

Part 0 — How to use this document	3
1 Part I — The problem in plain words	4
1.1 Trees	4
1.2 Why trees are not enough: reticulation	4
1.3 Level-1 networks: the class we generate	4
1.4 What “generate” means here: a GFlowNet	4
2 Part II — The mathematics you need	6
2.1 Sequence evolution and the likelihood of a tree	6
2.2 Priors and the reward	6
2.3 Likelihood of a network	6
2.4 GFlowNets and trajectory balance	7
2.5 Estimating the marginal likelihood	7
3 Part III — Anatomy of the original repository	8
3.1 The state and the action (<code>binary_tree_env_one_step_likelihood.py</code>)	8
3.2 The features (<code>batch_apply_actions, compute_partial_prob</code>)	9
3.3 The policy (<code>one_step_model.py</code>)	9
3.4 Branch lengths (<code>continuous_mixture.py</code>)	9
3.5 The TB loss (<code>tb_gfn_phylo.py</code>)	9
3.6 Rollouts and the number of parents (<code>rollout_worker_phylo.py</code>)	10
3.7 Replay buffer and backward sampling (<code>training_data_loader.py, sample_backward_from_tree</code>)	10
3.8 Evaluation (<code>gfn_evaluator.py</code>)	10

3.9	Training loop (<code>train.py</code>)	10
4	Part IV — Design of the network extension	11
4.1	The new MDP	11
4.2	The three masks and why there are no dead ends	11
4.3	Two features per lineage: the exact likelihood without 2^R trees	12
4.4	Ordering keys: keeping the list sorted so that actions are reproducible	13
4.5	Backward policy: counting parents correctly	13
4.6	Priors	13
4.7	The policy for two kinds of action	13
4.8	Variable-length batches	14
5	Part V — The modifications, one by one	15
5.1	M1 — Data structures: nodes with two parents	15
5.2	M2 — Initial state and the feature tensor	16
5.3	M3 — Level-1 masks	16
5.4	M4 — Transitions and ordering keys	17
5.5	M5 — Feature updates: the exact likelihood and the reward	17
5.6	M6 — Backward policy: counting parents, sampling backward	18
5.7	M7 — What the policy sees	19
5.8	M8 — The policy and the two continuous heads	19
5.9	M9 — Exact class sizes and the topology prior	19
5.10	M10 — The generator	20
5.11	M11 — Rollouts, replay buffer, evaluation	20
5.12	M12 — Registrations, configuration, scripts, tests	21
6	Part VI — Verification: what was tested and what the tests showed	22
6.1	(a) The class: exhaustive enumeration of the MDP	22
6.2	(b) The likelihood: two vectors versus 2^R displayed trees	22
6.3	(c) The backward policy	22
6.4	(d) Reduction to PhyloGFN	22
6.5	(e) Learning on simulated data, and the negative control	23
7	Part VII — How to run everything	25
7.1	Installation	25
7.2	Run the verification tests first	25
7.3	Simulate data with a known network and train	25
7.4	Train on a benchmark alignment	25
7.5	Tree mode (the equivalence check)	25
7.6	Configuration keys added	26
7.7	Reading a sampled network	26
8	Part VIII — What this prototype does not (yet) do	27
9	Part IX — Rehearsal: questions you will be asked	28
A	Glossary	31
B	Reference table of the tests	32

Part 0 — How to use this document

What your supervisor asked. “This code takes sequences and generates phylogenetic *trees*. Force it to generate phylogenetic *networks*.” A tree says every species has one parent lineage. A network allows a lineage to have *two* parent lineages (a *reticulation*: hybridisation, horizontal gene transfer, recombination). The code must therefore (i) be able to *build* such objects step by step, (ii) be able to *score* them with a likelihood, and (iii) keep the mathematical guarantee of a GFlowNet: sample each object with probability proportional to its score.

The one-sentence summary you must be able to say.

How to say it in the defence

PhyloGFN grows a tree by repeatedly *merging* two lineages under a new ancestor. I added one more move, *reticulate*, which puts a new hybrid node above a lineage and opens two parent “slots”; three constant-time legality rules keep every object a level-1 network; a lineage carrying an open hybrid node keeps two partial likelihoods instead of one, so the network likelihood is computed exactly without enumerating the 2^R displayed trees; the reward gets a prior $e^{-\lambda R}$ on the number of hybrid nodes; the trajectory-balance objective, the uniform backward policy and the marginal-likelihood estimator are unchanged in form and I verified the whole thing against exact enumerations.

Reading order. Part I is biology in plain words. Part II is the mathematics (only what is needed). Part III walks through the *original* code file by file so that you know what is being modified. Part IV explains the *design* of the extension (the decisions and why they are correct). Part V lists every modification with its code. Part VI shows the tests and their results. Part VII tells you how to run everything. Part VIII is honest about limits. Part IX is a rehearsal of questions a professor will ask.

Notation used everywhere.

N	number of taxa (leaves, observed sequences)
m (code: <code>m</code>)	number of sites (alignment columns); $ \Sigma = 4$ for DNA
$\mathbf{Y} = (\mathbf{Y}^1, \dots, \mathbf{Y}^m)$	the alignment; $\mathbf{Y}^i$ is column i
G	topology (tree or network); $\mathbf{b}$ branch lengths; $\boldsymbol{\theta}$ inheritance weights
$R = R(G)$	number of reticulation nodes of G ; $R_{\max}$ the budget (config <code>R_MAX</code>)
h , slots p_1, p_2	a reticulation node and its two parent “slots”
θ_h	probability that a site inherits through slot p_1 of h ($1 - \theta_h$ through p_2)
$P(\mathbf{Y} G, \mathbf{b}, \boldsymbol{\theta})$	likelihood; $P(\mathbf{b})$, $P(\boldsymbol{\theta})$, $P(G)$ priors; $\mathcal{R}(x)$ the GFlowNet reward
$s_0 \rightarrow s_1 \rightarrow \dots \rightarrow x$	a trajectory of the MDP; P_F , P_B forward / backward policies; Z partition function

Files delivered with this lecture.

- `phylogfn-net/` — the modified repository (original files untouched except four small registrations; all new code in new files).
- `phylogfn-net.patch` — the exact `git diff` against the original repository.
- `tests/test_network_env.py` — the five verification tests (Part VI).
- `tools/simulate_network_data.py`, `tools/compare_tree_mode.py` — data simulation and the tree-mode consistency check.

1 Part I — The problem in plain words

1.1 Trees

A *phylogenetic tree* is a diagram of ancestry. The leaves are the N things we observed (species, strains, genes), each with a DNA sequence of m letters. Every internal node is a hypothetical common ancestor; every edge (branch) has a *length* b_e measuring how much evolutionary change happened along it. A *rooted binary* tree has one root, every internal node has exactly two children, and there are $2N - 1$ nodes and $2N - 2$ edges. Two facts drive everything:

- There are $(2N - 3)!! = 1 \cdot 3 \cdot 5 \cdots (2N - 3)$ rooted binary trees on N labelled leaves (945 for $N = 6$; about 10^{31} for $N = 27$). We cannot look at all of them.
- Under a model of sequence evolution we can compute how probable the data are for a given tree — the *likelihood* — in time linear in N and m (Felsenstein’s pruning algorithm, Part II).

Bayesian phylogenetics wants the *posterior* $P(G, \mathbf{b} \mid \mathbf{Y}) \propto P(\mathbf{Y} \mid G, \mathbf{b}) P(\mathbf{b}) P(G)$: not the single best tree but the whole distribution, because the data rarely determine the tree completely.

1.2 Why trees are not enough: reticulation

Bacteria exchange genes horizontally; plants hybridise; viruses recombine. Then some part of a genome has *two* parents. A *phylogenetic network* keeps the tree picture but allows nodes with two incoming edges, called *reticulation nodes* (we write h). A reticulation node has one child and two parents; each parent edge carries an *inheritance weight*: a site (a column of the alignment) descends through the first parent with probability θ_h and through the second with probability $1 - \theta_h$.

Displayed trees

Delete, for every reticulation node h , one of its two parent edges, and then erase every node that is left with one parent and one child (joining its two edges and adding their lengths). The result is a tree, called a *displayed tree* of the network. A network with R reticulation nodes displays 2^R trees. The displayed-tree likelihood model (Jin, Nakhleh, Snir & Tuller 2006) says: *each site of the alignment evolved along one displayed tree*, chosen independently with probability $w(T \mid \theta) = \prod_h \theta_h^{[T \text{ keeps } p_1(h)]} (1 - \theta_h)^{[T \text{ keeps } p_2(h)]}$.

1.3 Level-1 networks: the class we generate

General networks are far too many and their likelihood is far too expensive. The thesis restricts to *level-1* networks: every cycle of the (undirected) network contains exactly one reticulation node, and different cycles share no vertex. Pictorially: a tree with a few independent “bubbles” (galls) hanging in it; each bubble is a cycle that starts at a tree node, splits into two paths, and closes at a reticulation node. Level-1 is the smallest class that models independent reticulation events, it is the class for which many algorithmic questions are tractable, and — crucially for this work — its likelihood can be computed *without* listing the 2^R displayed trees (Part IV).

Numbers you should know

Level-1 networks with N labelled leaves and at most 2 reticulations: 603 for $N = 4$, 11 460 for $N = 5$, 236 835 for $N = 6$, 5 273 100 for $N = 7$, 126 087 255 for $N = 8$ (Bouvel, Gambette & Mansouri 2020, and the direct enumeration shipped with this code). The new MDP reaches exactly these numbers (Part VI).

1.4 What “generate” means here: a GFlowNet

A GFlowNet (generative flow network) is a learned stochastic *constructor*. It starts from an empty object and applies actions one at a time until the object is complete; the probability of ending at a complete object x is trained to be proportional to a reward $\mathcal{R}(x)$. If $\mathcal{R}(x) = P(\mathbf{Y} \mid x)P(x)$ then

the constructor samples the Bayesian posterior. PhyloGFN’s constructor starts from N separate leaves and merges two lineages per step: after $N - 1$ merges one tree remains. Our constructor also merges, but may additionally *reticulate*: put a hybrid node above a lineage and open two parent slots that later merges will fill. After $N - 1 + 2R$ steps one network remains.

2 Part II — The mathematics you need

2.1 Sequence evolution and the likelihood of a tree

Jukes–Cantor (JC69). Along an edge of length b a nucleotide changes according to the 4×4 matrix $P(b) = \exp(Qb)$ with rate matrix $Q_{ii} = -1$, $Q_{ij} = 1/3$. Closed form: $P(b)_{ii} = \frac{1}{4} + \frac{3}{4}e^{-4b/3}$, $P(b)_{ij} = \frac{1}{4} - \frac{1}{4}e^{-4b/3}$. The code computes $P(b) = U \text{diag}(e^{Db}) U^{-1}$ from the eigen-decomposition (`decompJC`, `get_transition_matrices`). Two properties used later: rows of $P(b)$ sum to one ($P(b)\mathbf{1} = \mathbf{1}$), and $P(b_1)P(b_2) = P(b_1 + b_2)$ (Markov property).

Felsenstein pruning (paper eq. 1). For node u and site i let $L_u^i(a)$ be the probability of the data below u given that u carries nucleotide a . Leaves: $L_v^i(a) = [y_v^i = a]$ (one-hot). Internal node u with children v, w and edges b_v, b_w :

$$L_u^i(a) = \left(\sum_{a'} P(b_v)_{aa'} L_v^i(a') \right) \left(\sum_{a''} P(b_w)_{aa''} L_w^i(a'') \right), \quad \text{i.e. } L_u = (L_v P(b_v)^\top) \odot (L_w P(b_w)^\top). \quad (1)$$

The likelihood of site i is $P(\mathbf{Y}^i \mid G, \mathbf{b}) = \sum_a \pi_a L_{\text{root}}^i(a)$ with $\pi_a = \frac{1}{4}$, and $P(\mathbf{Y} \mid G, \mathbf{b}) = \prod_i P(\mathbf{Y}^i \mid G, \mathbf{b})$ (sites independent). In the code the vector over sites and nucleotides $L_u \in \mathbb{R}^{m \times 4}$ is called a *feature* of the (sub)tree rooted at u ; `compute_partial_prob` implements eq. (1) and returns both the new feature and the log-likelihood.

The linearity fact. Eq. (1) is *linear* in each child feature: if $L_v = \alpha L'_v + \beta L''_v$ then $L_u = \alpha L'_u + \beta L''_u$ with the same L_w . Everything in Part IV rests on this.

2.2 Priors and the reward

PhyloGFN (Sec. 4.1 of the paper) uses a uniform prior over topologies and independent exponential priors on branch lengths, $P(b_e) = \lambda e^{-\lambda b_e}$ with $\lambda = 10$ (config `PRIOR_LAMBDA`). The reward is the unnormalised posterior

$$\mathcal{R}(G, \mathbf{b}) = P(\mathbf{Y} \mid G, \mathbf{b}) P(\mathbf{b}), \quad \log \mathcal{R} = \log P(\mathbf{Y} \mid G, \mathbf{b}) + \sum_e \log P(b_e). \quad (2)$$

The code folds the prior into the features site by site: the feature of a subtree is multiplied by $P(b_e)^{1/m}$ for every edge e inside it (paper eq. 4, $\rho_u^i = f_u^i \prod_e P(b_e)^{1/m}$), so that the product over the m sites at the root reproduces $\prod_e P(b_e)$ exactly. Training uses a *temperature*: $\log \mathcal{R}_T = (C + \log \mathcal{R})/T$ with T annealed from 16 (or 8) down to 1; at $T = 1$ the target is the true posterior.

2.3 Likelihood of a network

Under the displayed-tree model,

$$P(\mathbf{Y} \mid G, \mathbf{b}, \boldsymbol{\theta}) = \prod_{i=1}^m \sum_{T \in \mathcal{T}(G)} w(T \mid \boldsymbol{\theta}) P(\mathbf{Y}^i \mid T, \mathbf{b}), \quad w(T \mid \boldsymbol{\theta}) = \prod_h \theta_h^{[T \ni p_1(h)]} (1 - \theta_h)^{[T \ni p_2(h)]}. \quad (3)$$

Note the order: the sum over displayed trees is *inside* the product over sites (each site chooses its own tree). The network reward adds a prior on the number of reticulations,

$$\log \mathcal{R}(G, \mathbf{b}, \boldsymbol{\theta}) = \log P(\mathbf{Y} \mid G, \mathbf{b}, \boldsymbol{\theta}) + \sum_e \log P(b_e) - \lambda_R R(G), \quad p(\theta_h) = \text{Uniform}(0, 1), \quad (4)$$

so that $p(G) \propto e^{-\lambda_R R(G)}$ over the level-1 class (λ_R is config `LAMBDA_R`; a hyper-prior on λ_R is future work, see Part VIII).

2.4 GFlowNets and trajectory balance

The objects

A GFlowNet is defined on a directed acyclic graph of *states*: an initial state s_0 , intermediate states, and *terminal* states x (the objects we want to sample). A *forward policy* $P_F(s' | s)$ is a distribution over the children of s ; following it from s_0 produces a trajectory $\tau = (s_0 \rightarrow s_1 \rightarrow \dots \rightarrow x)$ with probability $P_F(\tau) = \prod_t P_F(s_{t+1} | s_t)$. A *backward policy* $P_B(s | s')$ is a distribution over the parents of s' . The *partition function* is $Z = \sum_x \mathcal{R}(x)$.

The goal (paper eq. 2): $P_F^\top(x) := \sum_{\tau \rightarrow x} P_F(\tau) = \mathcal{R}(x)/Z$. The *trajectory balance* objective (Malkin et al. 2022; paper eq. 3) is, for one trajectory,

$$\mathcal{L}_{\text{TB}}(\tau) = \left(\log Z_\phi + \sum_t \log P_F(s_{t+1} | s_t; \phi) - \log \mathcal{R}(x) - \sum_t \log P_B(s_t | s_{t+1}) \right)^2. \quad (5)$$

If $\mathcal{L}_{\text{TB}} = 0$ on all trajectories, then $P_F^\top(x) = \mathcal{R}(x)/Z$ for every x , whatever P_B is (it only has to be a valid distribution over parents). PhyloGFN — and we — fix P_B *uniform*: $P_B(s | s') = 1/|\text{Pa}(s')|$, so the only thing the code has to know about the backward direction is *how many parents a state has*.

Why the number of parents matters so much

Every state of the tree MDP (a set of l rooted trees) can be reached from several parents: undo the last merge of any of its non-leaf trees. PhyloGFN counts these (`has_children.sum(-1)`) and uses $2N - 3$ at the last step, because the terminal object is an *unrooted* tree that can be produced by $2N - 3$ different last merges. In the network MDP a state can also be reached by undoing a reticulation, and only when the undo leads to a *legal* state; getting this count right is one of the modifications (M6).

Continuous actions. Branch lengths are real numbers, so a “step” is a discrete choice (which pair) *and* a continuous draw (the lengths). Then $P_F(s' | s)$ is a probability *density* and eq. (5) still holds (Lahlou et al. 2023). PhyloGFN-Continuous models $\log b$ by a Gaussian mixture and converts the density with the change of variables $\log p(b) = \log p(\log b) - \log b$ (`correction = edge_actions` in `compute_log_path_pf`). We reuse this and add the same trick for θ through its logit.

2.5 Estimating the marginal likelihood

The evidence $P(\mathbf{Y}) = \sum_G \int P(\mathbf{Y} | G, \mathbf{b}) P(\mathbf{b}) P(G) d\mathbf{b}$ is what MrBayes’ stepping-stone estimates. PhyloGFN’s estimator (paper eq. 5, Appendix B) samples K trajectories from P_F and computes the importance-weighted lower bound

$$\log P(\mathbf{Y}) \geq \mathbb{E} \log \left(P(G) \frac{1}{K} \sum_{k=1}^K \frac{P_B(\tau_k | x_k) \mathcal{R}(x_k)}{P_F(\tau_k)} \right). \quad (6)$$

For trees $P(G) = 1/(2N - 5)!!$ (uniform over unrooted topologies; `tree_factor` in `gfn_evaluator.py`). For networks $P(G) = e^{-\lambda_R R(G)} / Z_\lambda$ with $Z_\lambda = \sum_{k \leq R_{\max}} r(N, k) e^{-\lambda_R k}$, where $r(N, k)$ is the number of level-1 networks with N leaves and k reticulations — computed exactly from a published closed formula (M9).

3 Part III — Anatomy of the original repository

The repository is small (about 4 100 lines of Python). Table 1 maps every file to the part of the paper it implements. Read this table twice; the modifications of Part V are organised by the same files.

File	What it does (paper section)
<code>train.py</code>	Entry point: loads sequences and config, builds env / model / rollout worker / data loader / evaluator; epoch loop with ε and temperature schedules; checkpoints; MLL evaluation (Sec. 4.3, App. E).
<code>src/configs/defaults.py</code>	All hyper-parameters (yacs/fvcore CfgNode); YAML files in <code>src/configs/</code> override them.
<code>src/env/binary_tree_env_one_step_likelihood.py</code>	The MDP. State = list of rooted subtrees; action = (pair, edge lengths); features ρ ; reward; random rollouts; backward sampling with uniform P_B (Sec. 4.1, App. A).
<code>src/env/edges_env/continuous.py</code>	Random branch lengths and the action $\leftrightarrow$ length conversion for the continuous model (App. F).
<code>src/utils/evolution_model_torch.py</code>	JC69 transition matrices, exponential edge prior, one Felsenstein step (<code>compute_partial_prob</code>) (Sec. 3.1.1, eq. 4).
<code>src/model/transformer.py</code>	Transformer encoder (ViT-style blocks) over the set of lineage features (Sec. 4.3, App. D).
<code>src/model/tree_topologies_model/one_step_model.py</code>	The policy. Embeds features, encodes the set, builds all $\binom{l}{2}$ pair features $e_i + e_j$, MLP $\rightarrow$ logits, samples a pair (App. D).
<code>src/model/edges_model/continuous/continuous_mixture.py</code>	Edge MLP: Gaussian mixture over $\log b$ (one edge at the last step, two otherwise) (App. F).
<code>src/gfn/tb_gfn_phylo.py</code>	Trajectory balance. Holds model, edge model and $\log Z$; computes the TB loss; optimiser and clipping (Sec. 3.2, eq. 3).
<code>src/gfn/rollout_worker_phylo.py</code>	Runs a batch of trajectories in lock-step; accumulates $\log P_F$ per step and the number of parents for $\log P_B$; $2N - 3$ at the last step.
<code>src/gfn/training_data_loader.py</code>	Mini-batches = on-policy rollouts (ε -greedy) + trajectories re-sampled backward from the replay buffer of best trees (Sec. 4.3).
<code>src/gfn/gfn_evaluator.py</code>	Importance-weighted MLL (eq. 5) and Pearson correlation between $\log P_F^\top(x)$ and $\log \mathcal{R}(x)$ (Sec. 5.1).

Table 1: Files of the original repository and the paper sections they implement.

3.1 The state and the action (`binary_tree_env_one_step_likelihood.py`)

A state is `PhylogeneticTreeState(subtrees, max_root_idx, log_reward)`: `subtrees` is a Python list of `PhylogeneticTree` objects (thin wrappers around `ete3.TreeNode`). The initial state has N single-leaf trees (`get_initial_state`). The list is kept *sorted by min_seq_idx* (the smallest leaf index in each subtree): after merging positions $i < j$, the new subtree takes position i and position j disappears (`update_state`), which keeps the list sorted because the new subtree’s minimum is the minimum of subtree i . This invariant is what allows the backward sampler to recompute action indices from a finished tree.

An action is a dictionary `{‘tree_action’: k, ‘edge_action’: (b_l, b_r)}`: k indexes the list of all pairs (i, j) of the current l subtrees (`tree_pairs_dict[1]`, built once with `itertools.combinations`); the edge action is the pair of branch lengths of the two new edges (at the last step, where $l = 2$, a single length that is split in halves, because under a reversible model only the sum matters — the “pulley principle”).

3.2 The features (batch_apply_actions, compute_partial_prob)

Each subtree carries a feature tensor of shape $[m, 4]$: the Felsenstein vector L_u multiplied by $\prod_e P(b_e)^{1/m}$ (eq. 4 of the paper). The batch of trajectories keeps all features in one tensor `tree_features` of shape $[B, l, m, 4]$; `batch_apply_actions` gathers the left and right features of every trajectory, calls

```
1 merged_trees_features, log_scores = self.evolution_model.compute_partial_prob(
    data, at_root)
```

which applies eq. (1) to the whole batch at once, multiplies by the edge priors, and returns the log-likelihood `log_scores =  $\sum_i \log \sum_a L^i(a)/4$` (only meaningful at the root, but computed every step). The reward is `log_rewards = (C + log_scores) / scale (PhyloTreeReward)`.

Why the policy only sees features

Proposition 1 of the paper: two states with the same set of features have the same optimal forward policy, so the policy need not see the tree structures at all — only the l feature vectors. This is why the model input is a *set* of vectors and why a Transformer (permutation-equivariant) is used.

3.3 The policy (one_step_model.py)

```
1 x = self.seq_emb(batch_input) # [B, l, 4m] -> [B, l, 128]
2 x = torch.cat((summary_token, x), dim=1) # prepend a learned [CLS]-
    like token
3 x = self.encoder(x, batch_padding_mask) # Transformer encoder
4 summary_token, trees_reps = x[:, :1], x[:, 1:]
5 tmp = trees_reps[:, :, None, :] + trees_reps[:, None, :, :] # e_i + e_j for
    all i, j
6 row, col = torch.triu_indices(max_nb_seq, max_nb_seq, offset=1)
7 x_pairs = tmp[:, row, col] # all distinct pairs [B, l
    (l-1)/2, 128]
8 x_pairs = torch.cat([x_pairs, summary...], dim=2) # concatenate the summary
9 logits = self.logits_head(x_pairs).squeeze(-1) # one logit per pair
10 tree_actions = self.sample(ret, random_spec) # Categorical / epsilon-
    greedy / temperature
11 log_paths_pf = log_softmax(logits)[arange(B), tree_actions]
```

The pair feature is the *sum* $e_i + e_j$ (order-free), concatenated with the summary token (App. D). `sample` implements ε -greedy exploration (`random_action_prob`) or temperature sampling (T). If `input_tree_actions` is given (replay-buffer trajectories), the sampled actions are overwritten by the given ones and only their log-probability is computed.

3.4 Branch lengths (continuous_mixture.py)

`MixtureGaussianModel` maps $[\text{summary}; e_i; e_j]$ to the parameters of a Gaussian mixture over *log* branch lengths ($K = 5$ components; the means pass through $\tanh$ into $[-10, 0]$). `EdgeModelContinuousMixture` uses the two-dimensional (independent) model at intermediate steps and the one-dimensional `root_edge_model` at the last step; `compute_log_path_pf` subtracts $\log b$ (change of variables). Random exploration is not applied to branch lengths (App. F).

3.5 The TB loss (tb_gfn_phylo.py)

```
1 log_pf = log_paths_pf.sum(-1) # sum over the N-1 steps
2 log_pb = log_paths_pb.sum(-1) # = -sum log / Pa(s_t) /
3 log_z = self.compute_log_Z(None) # self._Z is a 256-vector; log Z = its
    sum (a trick for the LR)
4 forward_value = log_z + log_pf
```

```

5 backward_value = log_rewards + log_pb
6 loss = self.loss_fn(forward_value, backward_value)      # MSE == eq. (3)
    averaged over the batch

```

$\log Z$ is stored as 256 numbers whose sum is $\log Z$ (so that Adam’s per-parameter step is effectively 256 times larger); it is initialised from the best reward among 1000 random rollouts and gets its own learning rate. Temperature annealing rescales it (`current_log_z * (old_T / new_T)` in `train.py`), because $\log Z_T \approx \log Z_1/T$ when the reward is $\mathcal{R}^{1/T}$.

3.6 Rollouts and the number of parents (`rollout_worker_phylo.py`)

All B trajectories of a batch are advanced together (`while tree_features.shape[1] > 1`), because every tree trajectory has exactly $N - 1$ steps. After each step the worker records how many subtrees of the new state have children (`has_children.sum(-1)`): that is $|\text{Pa}(s')|$ under the “undo one merge” backward move. The last entry is overwritten with $2N - 3$: the terminal object is unrooted, and an unrooted tree with $2N - 3$ edges can be produced by $2N - 3$ different last merges (one per edge on which the root could sit).

3.7 Replay buffer and backward sampling (`training_data_loader.py`, `sample_backward_from_tree`)

The best trees seen so far (by `log_score`) are kept in a heap keyed by a signature (topology id + score). To train on one of them, the environment samples a trajectory *backward*: choose uniformly one of the $2N - 3$ edges as the root, then repeatedly pick uniformly a non-leaf subtree and split it (`sample_backward_prevsteps`); the forward action indices are recovered from the sorted-by-`min_seq_idx` convention; the edge actions are the stored branch lengths. The resulting action list is fed as `input_actions_set` to the rollout worker, which then only evaluates $\log P_F$ of the prescribed actions.

3.8 Evaluation (`gfn_evaluator.py`)

`evaluate_marginal_likelihood`: 1024 trajectories from P_F ; `logsumexp(log_scores + log_pb - log_pf) - log K + tree_factor` where `tree_factor` = $-\log(2N - 5)!!$ is $\log P(G)$ for the uniform prior over unrooted topologies (eq. (6)). `evaluate_gfn_quality_pearsonr`: for 100 fixed trees, estimate $\log P_F^\top(x)$ by importance sampling over backward paths (paper eq. S1) and correlate with $\log \mathcal{R}(x)$.

3.9 Training loop (`train.py`)

Per epoch: compute the temperature (linear or cascading schedule), rescale $\log Z$ if it changed; build the exploration schedule for ε ; for each of `STEPS_PER_EPOCH` batches: `generate_batch` (rollouts + buffer trajectories), `accumulate_loss`, `update_model`; then save a checkpoint, run the evaluator, and refresh the replay buffer.

4 Part IV — Design of the network extension

This part explains *what* was decided and *why it is correct*, before showing code. Everything here is implemented in `src/env/phylo_network_env.py`.

4.1 The new MDP

State

A state is a sorted list of *lineages*. A lineage is either

- a *real* lineage: the root node of a partial network (a leaf, a tree node, or a subnetwork), or
- a *bare parent slot* p_k ($k \in \{1, 2\}$) of a reticulation node h : a “hook” with no data of its own, waiting to be given a parent.

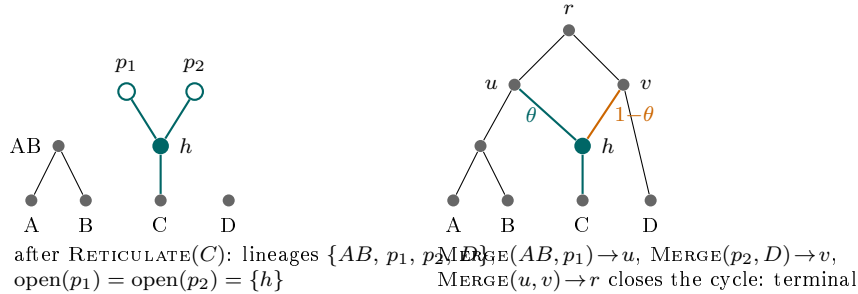
Each lineage u carries $\text{open}(u)$: the reticulation node below u (if any) whose two slots are not yet both below one lineage. Under our masks $|\text{open}(u)| \leq 1$.

Actions

$\text{MERGE}(i, j)$: create a tree node w with children lineages i and j and two sampled branch lengths (b_i, b_j) — *exactly* PhyloGFN’s action. If lineage i is a bare slot of h , the edge $w \rightarrow h$ is created with length b_i (the slot is “filled”).

$\text{RETICULATE}(i)$: create a reticulation node h above real lineage i (edge $h \rightarrow i$ with sampled length b_{hx}), sample $\theta_h \in (0, 1)$, and replace lineage i by two bare slots p_1, p_2 of h . The number of lineages increases by one.

Terminal state: one lineage, real, with $\text{open} = \emptyset$. No explicit STOP is needed: the state is terminal by definition when this happens.



Trajectory length. Each MERGE removes one lineage, each RETICULATE adds one; from N lineages to 1 with R reticulations takes $N - 1 + R$ merges and R reticulate actions: $N - 1 + 2R$ steps (checked by `test_trajectory_length_and_reward`).

4.2 The three masks and why there are no dead ends

Legality (constant time per action)

- $\text{MERGE}(u, v)$ is legal iff $|\text{open}(u) \cup \text{open}(v)| \leq 1$ and (u, v) are not the two bare slots of the same h .
- $\text{RETICULATE}(x)$ is legal iff $\text{open}(x) = \emptyset$, fewer than $R_{\max}$ reticulations exist, and some *other* lineage $y \neq x$ has $\text{open}(y) = \emptyset$.

Why the first rule. Merging two lineages that carry two different open reticulations $h \neq h'$ would create a node through which two cycles pass; in a binary network two cycles that share a vertex necessarily share an edge (a vertex of degree 3 cannot host two edge-disjoint cycles), which is level-2. Merging the two bare slots of one h would create two parallel edges into h .

Why the second rule (no dead ends). After $\text{RETICULATE}(x)$ the slots p_1, p_2 cannot merge with each other and cannot merge with a lineage carrying a different open reticulation; they need a *free* lineage (open = $\emptyset$). The state $\{p_1, p_2, q_1, q_2\}$ (two bare pairs, nothing else) has no legal action at all. Requiring another free lineage at the moment of reticulating makes such states unreachable: by induction, every reachable non-terminal state has at least one legal action (if two free lineages exist, merge them; if one free and a pair, merge the free one into a slot; if only pairs, at least one pair is not bare, and its two carriers can close). The test `test_class_counts_and_no_dead_ends` enumerates *all* reachable states for $N \leq 5$ and finds no dead end.

Why this is exactly the level-1 class. Every terminal network is binary, acyclic (a new node is always older than what it connects) and level-1 (each cycle closes at exactly one reticulation and cycles are vertex-disjoint by the first rule). Conversely every level-1 network is reachable (induction on R : remove one cycle, build the rest, reticulate at the right moment). The enumeration in Part VI confirms both directions numerically: the reachable terminal networks are $603 = 15 + 228 + 360$ for $N = 4, R \leq 2$ and $11\,460 = 105 + 2\,805 + 8\,550$ for $N = 5$, the published counts.

4.3 Two features per lineage: the exact likelihood without 2^R trees

The trick

A lineage u that carries an open reticulation h stores two Felsenstein features:

- A_u = feature of u in the displayed trees where u 's side *keeps* its edge into h ;
- B_u = feature of u in the displayed trees where that edge is *deleted*.

For a bare slot: $A_{p_1} = \theta_h L_h$, $A_{p_2} = (1 - \theta_h) L_h$, $B_{p_1} = B_{p_2} = \mathbf{1}$ (all-ones: “no data below”).

$\text{MERGE}(u, v) \rightarrow w$ with lengths (b_u, b_v) , writing $T_u(\cdot) = (\cdot) P(b_u)^\top$:

both closed: $A_w = T_u(A_u) \odot T_v(A_v)$;

only u open (w stays open): $A_w = T_u(A_u) \odot T_v(A_v)$, $B_w = T_u(B_u) \odot T_v(A_v)$;

closing merge (w closed): $A_w = T_u(A_u) \odot T_v(B_v) + T_u(B_u) \odot T_v(A_v)$.

Edge priors $P(b)^{1/m}$ multiply both versions, exactly as in PhyloGFN.

Proposition 1. *For every level-1 network built by the MDP, the feature A of the final lineage equals, at every site i , $\sum_{T \in \mathcal{T}(G)} w(T \mid \boldsymbol{\theta}) L_{\text{root}}^i(\cdot \mid T, \mathbf{b})$ (times the edge priors). Hence $\sum_a \frac{1}{4} A^i(a)$ is the network likelihood of site i (eq. (3)).*

Proof sketch. Fix a reticulation h with carriers u (slot 1 side) and v (slot 2 side) at the closing merge. The displayed trees split into those keeping p_1 (weight $\theta_h \cdot \text{rest}$) and those keeping p_2 . In a tree keeping p_1 , the edge $v \rightarrow h$ is deleted and the slot is suppressed, so v contributes its feature *without* the h subtree: that is B_v (the all-ones vector of a bare slot is invariant under T_v because $P(b)\mathbf{1} = \mathbf{1}$, and suppressing a degree-2 node is exactly $P(b_1)P(b_2) = P(b_1 + b_2)$). Symmetrically for trees keeping p_2 . Because pruning is linear in each child feature and $\theta_h, 1 - \theta_h$ were folded into A_{p_1}, A_{p_2} , the sum of the two products is the θ -weighted sum over the two families of displayed trees. Cycles of a level-1 network are vertex-disjoint, so the families of different reticulations are independent and the argument applies to each closing merge in turn; nested (closed) cycles enter as already-mixed features, by linearity. $\square$

The test `test_likelihood_matches_explicit` compares this with an explicit enumeration of all 2^R displayed trees (`src/Utils/network_likelihood.py`) on random networks with up to 7 leaves and 3 reticulations; the largest difference is 1.5×10^{-11} .

Why this matters for scalability

The naive network likelihood costs 2^R Felsenstein passes; ours costs *one* pass with at most two vectors per lineage — $O(Nm|\Sigma|^2)$ per trajectory, the same order as a tree. This is what makes reward evaluation compatible with the “reward is the dominant per-trajectory cost” analysis of the proposal.

4.4 Ordering keys: keeping the list sorted so that actions are reproducible

PhyloGFN sorts subtrees by their smallest leaf index. A bare slot has no leaves, and the leaves below h would be counted by *both* sides of h . We therefore give every lineage a set of *tokens*: the leaf indices $0, \dots, N-1$, plus one fresh token $N + \text{id}(h)$ for slot 2 of every reticulation; slot 1 inherits the tokens of h ’s child. Tokens partition a set, so the minimum token (the *key*) of each lineage is unique and the sorted-by-key list is well defined. $\text{MERGE}(i < j)$ places the new lineage at position i ; $\text{RETICULATE}(i)$ places p_1 at position i and appends p_2 (its fresh token is larger than every other key). The backward sampler recomputes action indices by replaying forward under this convention (`test_backward_sampling_roundtrip`).

4.5 Backward policy: counting parents correctly

$|\text{Pa}(s')| = (\text{number of real lineages whose root is a tree node: undo a merge}) + (\text{number of reticulations whose two slots are both bare in } s', \text{ provided } s' \text{ contains a free lineage: undo a reticulation})$. The proviso is essential: undoing $\text{RETICULATE}(x)$ leads to a state from which $\text{RETICULATE}(x)$ is legal only if another free lineage exists; without the proviso we would count a “parent” that is not in the state graph, and P_B would not be a distribution over actual parents — the TB identity would be silently wrong. At the terminal state (rooted network) there is exactly one parent, the un-merge of the root; we keep the terminal object *rooted* (reticulations are directed, so the root is meaningful), unlike PhyloGFN’s unrooted trees with $2N - 3$ last-step parents.

4.6 Priors

$p(\mathbf{b}) = \prod_e \lambda e^{-\lambda b_e}$ with PhyloGFN’s root convention (the two root edges count as one edge of length $b_l + b_r$); $p(\theta_h) = \text{Uniform}(0, 1)$. For the topology two priors are implemented (config `PRIOR_TYPE`):

- `EXP_R` (the proposal’s formula): $p(G) \propto e^{-\lambda_R R(G)}$ over the level-1 class, $R \leq R_{\max}$;
- `UNIFORM_R`: $p(R) \propto e^{-\lambda_R R}$ and, given R , uniform over the $r(N, R)$ topologies: $\log p(G) = -\lambda_R R - \log r(N, R) + \text{const}$.

The difference is not cosmetic. The class sizes grow fast with R : for $N = 6$, $r(6, 1)/r(6, 0) = 41.6$ and $r(6, 2)/r(6, 1) = 5.0$; for $N = 27$ the ratios are in the thousands. Under `EXP_R` the prior mass of “some R ” is $r(N, R)e^{-\lambda_R R}$, so with $\lambda_R = 1$ a 6-taxon prior already favours $R = 2$ over $R = 1$ by $1.8 : 1$ and $R = 1$ over $R = 0$ by $15 : 1$ *before seeing any data*. `UNIFORM_R` removes the class-size effect and makes λ_R a genuine penalty per reticulation ($p(R) \propto e^{-\lambda_R R}$). Both are exact in the code; the normaliser needed for the MLL estimate uses the exact counts $r(N, k)$ in the first case and $\sum_k e^{-\lambda_R k}$ in the second. Because $\log p(G)$ is added to $\log \mathcal{R}$, the sampler’s marginal over R is a genuine posterior over the number of reticulations under the chosen prior.

4.7 The policy for two kinds of action

The Transformer sees, per lineage, $[\bar{A}; \bar{B}; 4 \text{ flags}]$ where $\bar{A}, \bar{B}$ are the per-site normalised features (as PhyloGFN normalises ρ) and the flags are (has open reticulation, is bare slot, which side of h , its inheritance share θ or $1 - \theta$). The flags let the policy recognise the two carriers of the same h (their A vectors share the h -subtree pattern). The action logits are the concatenation [pair logits | single logits]: pairs from the PhyloGFN head on $e_i + e_j$, singles from a new head on e_i ; illegal entries are set to $-\infty$ using the environment’s mask; ϵ -exploration picks a uniformly random

legal action. A RETICULATE action then draws $(\log b_{hx}, \text{logit } \theta_h)$ from a 2-D Gaussian mixture (new head) with the change of variables $\log p(\theta) = \log p(\text{logit } \theta) - \log \theta - \log(1 - \theta)$.

4.8 Variable-length batches

Trajectories in a batch no longer share a step index, so the rollout worker keeps a **done** mask, runs the policy only on active trajectories, pads the lineage dimension to $l_{\max} = N + R_{\max}$ (the Transformer already supports padding masks) and accumulates $\sum \log P_F$ and $\sum \log P_B$ per trajectory as it goes. The TB loss is then identical in form to PhyloGFN's.

5 Part V — The modifications, one by one

Principle. The original files are left untouched except for four small *registrations* (M12). Every new behaviour lives in new files whose names end in `_net` or `network`, so that `git diff` shows exactly what was added and the tree sampler keeps working unchanged (`ENVIRONMENT_TYPE: ONE_STEP_BINARY_TREE`). Table 2 is the map.

Id	File (new unless marked)	Content
M1	<code>src/env/phylo_network_env.py</code>	node / lineage / state / network data structures
M2	<code>src/env/phylo_network_env.py</code>	initial state and the two-version feature tensor
M3	<code>src/env/phylo_network_env.py</code>	level-1 masks
M4	<code>src/env/phylo_network_env.py</code>	transitions <code>MERGE</code> / <code>RETICULATE</code> , ordering keys
M5	<code>src/env/phylo_network_env.py</code>	feature updates = exact likelihood, reward
M6	<code>src/env/phylo_network_env.py</code>	number of parents, backward sampling from a network
M7	<code>src/env/phylo_network_env.py</code>	policy inputs: normalisation, flags, masks, padding
M8	<code>src/model/network_model/one_step_network_model.py</code> , <code>src/model/edges_model/continuous/network_heads.py</code>	policy with two action types; heads for lengths and θ
M9	<code>src/utils/level1_counts.py</code>	exact class sizes; topology prior normaliser
M10	<code>src/gfn/tb_gfn_net.py</code>	generator: routing of actions, TB loss
M11	<code>src/gfn/rollout_worker_net.py</code> , <code>src/gfn/training_data_loader_net.py</code> , <code>src/gfn/gfn_evaluator_net.py</code>	variable-length rollouts, replay buffer, evaluation
M12	<code>src/configs/defaults.py</code> , <code>src/env/__init__.py</code> , <code>src/gfn/build.py</code> , <code>src/utils/utils.py</code> (modified); <code>train_net.py</code> , <code>src/configs/network_cfgs/*.yaml</code> , <code>tools/*</code> , <code>tests/*</code>	registrations, config, training script, data simulation, tests

Table 2: Where each modification lives.

5.1 M1 — Data structures: nodes with two parents

What. PhyloGFN stores trees as `ete3.TreeNode`, which has exactly one parent per node. A reticulation node needs two. `NetNode` is a minimal DAG node; `Lineage` is an entry of the state list (a real root, or a bare slot); `NetworkState` replaces `PhylogeneticTreeState`; `PhyloNetwork` replaces `PhylogeneticTree` (same interface: `log_score`, `signature`, ordering for the replay heap).

```

1 class NetNode(object):
2     """kind: 'leaf' / 'tree' / 'ret'
3         ret: one child (child_dist), two parents = two SLOTS parents[k],
4             parent_dists[k];
5             theta = inheritance weight of slot 0, 1 - theta of slot 1."""
6     __slots__ = ('idx', 'kind', 'children', 'child_dists', 'parents', 'parent_dists',
7                 'theta', 'child_dist', 'tokens', 'ret_id')
8
9 class Lineage(object):
10     """node: NetNode; slot: None (real lineage) or 0/1 (bare parent slot of
11         node = h);
12     open_h: the unclosed reticulation carried (None if closed);

```

```

11         tokens: ownership tokens; key = min(tokens) (unique -> sorted state list
12             ). """
13     class NetworkState(object):
14         def __init__(self, lineages, next_idx, n_ret, log_reward=None, log_score=
15             None):
16             self.lineages = lineages; self.next_idx = next_idx; self.n_ret = n_ret
17             self.is_done = (len(lineages) == 1 and lineages[0].open_h is None)

```

Why. Everything downstream (transitions, likelihood, backward sampling, canonical signatures) needs to know, for a reticulation, *which* parent is which and what θ belongs to which slot. `network_canonical_string` writes a reticulation subtree as `#<subtree>` at both parent positions; on level-1 networks with labelled leaves this string is a faithful topology identifier (used by the replay buffer instead of `ete3`'s `get_topology_id`).

5.2 M2 — Initial state and the feature tensor

What. `get_initial_state` creates N leaf lineages with token sets $\{0\}, \dots, \{N-1\}$. `initial_features(B)` returns a tensor of shape $[B, l_{\max}, 2, m, 4]$ (two versions per lineage) filled with ones, with the one-hot sequences copied into *both* versions of the first N rows.

```

1 self.l_max = self.n_leaves + self.r_max # a RETICULATE adds one
   lineage
2 rows, cols = torch.triu_indices(self.l_max, self.l_max, offset=1)
3 self.n_pairs = len(rows); self.n_actions = self.n_pairs + self.l_max # [
   pairs | singles]
4
5 def initial_features(self, batch):
6     feats = torch.ones(batch, self.l_max, 2, self.m, self.c, dtype=..., device
   =...)
7     feats[:, :self.n_leaves, 0] = self.seq_arrays
8     feats[:, :self.n_leaves, 1] = self.seq_arrays
9     return feats

```

Why. PhyloGFN's tensor is $[B, l, m, 4]$ with l shrinking by one every step for *all* trajectories. Ours must (i) hold two versions and (ii) keep a fixed width $l_{\max} = N + R_{\max}$ because the number of lineages differs between trajectories of the same batch; unused rows are ones and are hidden from the policy by a padding mask (M7). The action space is laid out once, over $l_{\max}$: pair $k \mapsto (\text{rows}[k], \text{cols}[k])$, single $n_{\text{pairs}} + i \mapsto \text{lineage } i$.

5.3 M3 — Level-1 masks

What. `legal_actions(state)` returns a boolean vector over the padded action space; `merge_legal(u, v)` is the pair rule.

```

1 def legal_actions(self, state):
2     L = state.lineages; n = len(L)
3     mask = np.zeros(self.n_actions, dtype=bool)
4     if state.is_done: return mask
5     for i in range(n):
6         for j in range(i + 1, n):
7             mask[self.pair_index[(i, j)]] = self.merge_legal(L[i], L[j])
8     n_free = sum(1 for l in L if l.open_h is None)
9     if state.n_ret < self.r_max:
10        for i in range(n):
11            if L[i].open_h is None and n_free >= 2: # x free AND another
               free lineage exists
12                mask[self.n_pairs + i] = True
13    return mask
14
15 @staticmethod
16 def merge_legal(u, v):

```

```

17     if u.open_h is not None and v.open_h is not None:
18         if u.open_h is not v.open_h: return False      # two different open
19             cycles: level-2
20         if u.is_bare and v.is_bare: return False      # the two bare slots of
21             one h: parallel edges
22     return True

```

Why. See Part IV.2. The three rules are $O(1)$ per candidate action because $\text{open}(u)$ is stored on the lineage, never recomputed from the graph. In PhyloGFN every pair is legal, so the only mask is the padding mask; here the mask is also the definition of the class.

5.4 M4 — Transitions and ordering keys

What. `apply_action(state, action, feats)` implements both actions on the Python state and (optionally) on the feature tensor; it returns `(new_state, new_feats, log_score)` with `log_score` set only at a terminal state. The MERGE branch is PhyloGFN’s `update_state` plus the slot bookkeeping; the RETICULATE branch is new.

```

1  # MERGE(i, j): new tree node w; a bare slot attaches the reticulation node
   itself
2  for lin, d, slot_side in ((u, bl, 0), (v, br, 1)):
3      child = lin.node
4      w.children.append(child); w.child_dists.append(d)
5      if lin.is_bare:                      # fill the slot: edge w -> h
6          of length d
7          child.parents[lin.slot] = w; child.parent_dists[lin.slot] = d
8  open_h = None if case in (0, 3) else (u.open_h if case == 1 else v.open_h)
9  new_lin = Lineage(w, open_h=open_h, tokens=u.tokens | v.tokens)
10 new_L = L[:i] + [new_lin] + L[i + 1:j] + L[j + 1:]      # result at position i,
   j removed
11 # RETICULATE(i): h above lineage x, two bare slots p1 (position i) and p2 (
   appended)
12 h = NetNode(state.next_idx, 'ret'); h.children = [x.node]; h.child_dist = b_hx;
   h.theta = theta
13 fresh = self.n_leaves + state.n_ret                      # ordering token of
   slot 1
14 p1 = Lineage(h, slot=0, open_h=h, tokens=x.tokens)
15 p2 = Lineage(h, slot=1, open_h=h, tokens=[fresh])
16 new_L = L[:i] + [p1] + L[i + 1:] + [p2]

```

Why. `case` (0 both closed, 1 only u open, 2 only v open, 3 closing) decides both the open status of the result and the feature rule (M5). The token discipline replaces `min_seq_idx`: keys are unique and the list stays sorted, which is what the backward sampler needs to reconstruct the same action indices (Part IV.4).

5.5 M5 — Feature updates: the exact likelihood and the reward

What. Three private methods replace `compute_partial_prob`’s role for networks.

```

1  def _merge_features(self, feats, i, j, case, bl, br, last_step):
2      uA, uB = feats[i, 0], feats[i, 1]; vA, vB = feats[j, 0], feats[j, 1]
3      M = self.evolution_model.get_transition_matrices(torch.tensor([bl, br]))
4      if last_step:          # PhyloGFN root convention: one prior on b_l + b_r (
5          pulley principle)
6          tu = lambda f: f @ M[0]; tv = lambda f: f @ M[1]; prior = exp(log_prior
7              (bl + br) / m)
8      else:                  # edge priors folded into the features, site by site (
9          paper eq. 4)
10         pl = exp(log_prior([bl, br]) / m)
11         tu = lambda f: (f @ M[0]) * pl[0]; tv = lambda f: (f @ M[1]) * pl[1];
12         prior = 1.0

```

```

9     if case == 0: wA = tu(uA) * tv(vA) * prior; wB = wA
10    elif case == 1: wA = tu(uA) * tv(vA) * prior; wB = tu(uB) *
        tv(vA) * prior
11    elif case == 2: wA = tu(uA) * tv(vA) * prior; wB = tu(uA) *
        tv(vB) * prior
12    else: wA = (tu(uA) * tv(vB) + tu(uB) * tv(vA)) * prior; wB = wA
        # closing merge
13    ... row j removed, row i <- (wA, wB), a row of ones appended to keep l_max
        rows
14
15    def _reticulate_features(self, feats, i, n, theta, b_hx):
16        Lh = self._transport(feats[i, 0][None], torch.tensor([b_hx]))[0] # M(b_hx
        ) L_x * prior^(1/m)
17        new[i, 0] = theta * Lh; new[i, 1] = ones # slot 0 at
        position i
18        new[n, 0] = (1 - theta) * Lh; new[n, 1] = ones # slot 1 appended
        at position n
19
20    def log_topology_prior_unnormalised(self, n_ret):
21        if self.prior_type == 'UNIFORM_R': # p(R)
        ~ exp(-λR), uniform within R
22        return -self.lambda_r * n_ret - self.log_class_size[n_ret]
23        return -self.lambda_r * n_ret #
        EXP_R: p(G) ~ exp(-λR) over the class
24
25    def _terminal_log_score(self, root_feat, n_ret):
26        log_p = torch.sum(torch.log(torch.sum(root_feat / 4.0, -1)), -1) # sum_i
        log sum_a A^i(a)/4
27        return float(log_p) + self.log_topology_prior_unnormalised(n_ret) # + log
        p(G)

```

Why. This is Proposition 1 in code. Note what did *not* change: the transition matrices, the edge prior, the per-site folding of $P(b)^{1/m}$, the root formula $\sum_i \log \sum_a L^i(a)/4$ and the last-step convention are all PhyloGFN's — with $R_{\max} = 0$ the code computes exactly PhyloGFN's reward (Part VI.4). The reward reshaping $(C + \log_score)/scale$ is reused from PhyloTreeReward.

5.6 M6 — Backward policy: counting parents, sampling backward

What. `NetworkState.num_parents()` and `PhyloNetworkEnv.sample_backward_from_network(net)`.

```

1    def num_parents(self):
2        n = 0; seen_ret = {}; n_free = 0
3        for l in self.lineages:
4            if l.is_bare: seen_ret[l.node.idx] = seen_ret.get(l.node.idx, 0) + 1
5            else:
6                if l.open_h is None: n_free += 1
7                if l.node.kind == 'tree': n += 1 # un-merge
8        if n_free >= 1:
9            n += sum(1 for c in seen_ret.values() if c == 2) # un-reticulate
        (only if legal forward)
10        return n

```

`sample_backward_from_network` walks backward from the finished network: at every step it lists the undo moves (un-merge any tree-root lineage; un-reticulate any h whose two slots are both bare, provided a free lineage exists), records their number ($= |\text{Pa}|$), picks one uniformly, and stops at the leaves. It then *replays forward*, matching the original nodes to freshly created ones (`new_of`) to recover the pair / lineage indices under the sorted convention, and attaches the stored lengths and θ 's. It returns `(actions, log_paths_pb)` exactly like `sample_backward_from_tree`.

Why. Uniform P_B requires the true number of parents in the state graph (Part IV.5); the replay-buffer training and the Pearson- r evaluation both need backward-sampled trajectories of a *given* network. `test_backward_sampling_roundtrip` checks that the replay yields the same canonical network, the same score and the same $|\text{Pa}|$ sequence as the forward code.

5.7 M7 — What the policy sees

What. `prepare_rollout_inputs(feats, states, random_spec, input_actions)` normalises both versions per site (as PhyloGFN’s `NORMALIZE_LIKELIHOOD`), flattens them, appends four flags per lineage, and returns the padding mask, the action mask and the pair index tables.

```

1 x = feats / feats.sum(-1, keepdim=True); x[isnan] = 1/4          # per-site
   normalisation of A and B
2 x = x.reshape(b, self.l_max, -1)                                # [b, l_max,
   2*m*4]
3 flags[k, i] = [has_open, is_bare, side (slot 0 / slot 1), share (theta or 1 -
   theta)]
4 x = torch.cat([x, flags], dim=-1)
5 pad = torch.arange(self.l_max)[None] >= nb[:, None]            # True =
   padding
6 masks[k] = self.legal_actions(s)                                 # True =
   legal

```

Why. The Transformer needs a set of fixed-size vectors; padding rows are excluded by `key_padding_mask`, illegal actions by the action mask. The flags give the policy the information the masks use (Part IV.7); the environment — not the network — remains the single authority on legality.

5.8 M8 — The policy and the two continuous heads

What. `PhyloNetworkModelOneStep` keeps PhyloGFN’s encoder and pair head and adds `single_logits_head`; `MergeEdgeModel` serves root-mode and two-edge-mode merges in one batch; `ReticulationModel` samples ($\log b_{hx}$, $\text{logit } \theta$).

```

1 pairs = reps[:, rows] + reps[:, cols]                            # e_i + e_j (
   PhyloGFN)
2 singles = reps                                                    # e_i (new)
3 pair_logits = self.pair_logits_head(cat[pairs, summary]).squeeze(-1)
4 single_logits = self.single_logits_head(cat[singles, summary]).squeeze(-1)
5 logits = torch.cat([pair_logits, single_logits], dim=1).masked_fill(~
   action_mask, -inf)
6 actions = self.sample(logits, action_mask, random_spec)          # epsilon-greedy over
   LEGAL actions
7 log_paths_pf = log_softmax(logits)[arange(B), actions]

```

```

1 class ReticulationModel(nn.Module):                               # 2-D diagonal Gaussian mixture over
   (log b_hx, logit theta)
2 def forward(self, summary_reps, lineage_reps, input_ret_actions=None):
3     dist = self.distribution(torch.cat([summary_reps, lineage_reps], 1))
4     z = dist.sample(); z = [z0.clip(-10, 0), z1.clip(-r-2, r+2)]
5     lp = dist.log_prob(z)
6     b_hx, theta = exp(z0), sigmoid(z1)
7     lp = lp - z0 - log(theta) - log(1 - theta)                    # change of
   variables for b and theta
8     return b_hx, theta, lp

```

Why. The discrete choice and the continuous draw are two factors of one transition density; TB needs $\log P_F$ of the *whole* step, so the head’s log-density (with the Jacobian terms) is added to the discrete log-probability in the generator (M10). Squashing the mean of $\log b$ into $[-10, 0]$ is PhyloGFN’s trick (App. F); the analogous squash of $\text{logit } \theta$ into $[-6, 6]$ keeps θ away from the numerically degenerate ends 0 and 1 at initialisation.

5.9 M9 — Exact class sizes and the topology prior

What. `level1_counts.py` implements the closed formula of Bouvel, Gambette & Mansouri (2020, Prop. 5.4) for $r(n, k)$, the number of rooted binary level-1 networks with n labelled leaves and k retic-

ulations, and the prior normaliser $\log Z_\lambda = \log \sum_{k \leq R_{\max}} r(n, k) e^{-\lambda R k}$ (EXP_R) or $\log \sum_{k \leq R_{\max}} e^{-\lambda R k}$ (UNIFORM_R); the environment also uses $\log r(N, \bar{R})$ in the reward under UNIFORM_R.

```

1 def log_topology_prior_normalizer(n, r_max, lambda_r):
2     terms = [log(level1_count(n, k)) - lambda_r * k for k in range(0, r_max +
3         1)]
4     mx = max(terms); return mx + log(sum(exp(t - mx) for t in terms))

```

Why. It is the network analogue of PhyloGFN’s `tree_factor =  $-\log(2N - 5)!!$` in the MLL estimator: without it the “MLL” would be off by the log of the prior normaliser. The formula was checked against the paper’s Table 2 totals (36, 723, 20 280, 730 755) and against our own enumeration for $N \leq 6$; exact integers, no floating point.

5.10 M10 — The generator

What. `TBGFlowNetNet.forward(input_dict)` runs the policy, then routes the sampled ids: merges go to `MergeEdgeModel`, reticulations to `ReticulationModel`; the log-densities are added with `index_add`; the environment-format action dicts are assembled. Replay trajectories arrive as `input_actions` (one dict or None per trajectory) and are converted into forced ids and forced parameters (`nan` = not forced).

```

1 is_merge = actions < env.n_pairs
2 sel = where(is_merge); edges, lp = self.edges_model(summary[sel], left, right,
3     last_step[sel], edge_given[sel])
4 log_pf = log_pf.index_add(0, sel, lp)
5 sel = where(~is_merge); b_hx, theta, lp = self.ret_model(summary[sel], reps[sel],
6     lineage[sel], ret_given[sel])
7 log_pb = log_pb.index_add(0, sel, lp)
8 # loss (unchanged form): (log Z + log_pf - log_reward - log_pb)^2

```

Why. PhyloGFN’s generator always calls the edge model for *every* row of the batch; ours must split rows by action type. The TB loss, $\log Z$ parametrisation (256-vector), optimiser groups, gradient clipping and temperature rescaling are copied unchanged.

5.11 M11 — Rollouts, replay buffer, evaluation

Rollout worker.

```

1 done = np.array([s.is_done for s in states])
2 while not done.all():
3     active = np.where(~done)[0]
4     input_dict = env.prepare_rollout_inputs(feats[active], act_states,
5         random_spec, input_actions)
6     ret = generator(input_dict)
7     new_states, new_feats, ls, lr = env.batch_apply_actions(ret['actions'],
8         feats[active], act_states)
9     feats[active] = new_feats
10    log_pf = log_pf.index_add(0, active, ret['log_paths_pf'])
11    log_pb = log_pb.index_add(0, active, -log(tensor([s.num_parents() for s in
12        new_states])))
13    ... mark finished trajectories, store log_rewards / log_scores / n_ret

```

Why. Lock-step batching (`while tree_features.shape[1] > 1`) is impossible with variable lengths; the `done` mask and per-trajectory accumulation are the minimal change. The parent count of the *new* state is used, as in PhyloGFN; the last step now has one parent instead of $2N - 3$ (rooted terminal object).

Replay buffer (`training_data_loader_net.py`): same heap logic, keyed by the canonical network signature; backward trajectories from `sample_backward_from_network`.

Evaluator (`gfn_evaluator_net.py`): eq. (6) with $-\log Z_\lambda$ from M9, plus the histogram of sampled R (the posterior over the number of reticulations) — the quantity the validation plan calls “posterior mode R ” (T8).

5.12 M12 — Registrations, configuration, scripts, tests

Modified original files (the entire diff to the original code).

```
1 # src/env/_init_.py
2 if cfg.ENV.ENVIRONMENT_TYPE == 'ONE_STEP_LEVEL1_NETWORK': env = PhyloNetworkEnv
   (cfg, all_seqs)
3 # src/gfn/build.py
4 if cfg.ENV.ENVIRONMENT_TYPE == 'ONE_STEP_LEVEL1_NETWORK': generator =
   TBGFlowNetNet(cfg.GFN, env, device, ddp)
5 # src/utls/utls.py (correct_cfg_data): input size of the policy embedding
6 cfg.GFN.MODEL.TRANSFORMER.SEQ_EMB.INPUT_SIZE = 2 * seq_length * vocab_size + 4
7 # src/configs/defaults.py: ENV.NETWORK.{R_MAX, LAMBDA_R, PRIOR_TYPE},
   TRANSFORMER.SINGLE_LOGITS_HEAD, RETICULATION_MODELING
```

New scripts. `train_net.py` (training loop without DDP/AMP; same schedules), `src/configs/network_cfgs/cfg_net_small_cpu.yaml` (a CPU-sized configuration), `tools/simulate_network_data.py` (alignments simulated on a known level-1 network; writes the truth), `tools/compare_tree_mode.py` ($R_{\max} = 0$ versus PhyloGFN), `tests/test_network_env.py` (Part VI).

6 Part VI — Verification: what was tested and what the tests showed

The standard

A modification of a sampler is only worth presenting if (a) the object class is exactly the intended one, (b) the reward is exactly the intended likelihood, (c) the backward policy is exactly a distribution over parents, and (d) the whole thing reduces to the original when the new feature is switched off. Each of the four has an executable test.

6.1 (a) The class: exhaustive enumeration of the MDP

`tests/test_network_env.py::test_class_counts_and_no_dead_ends` runs a depth-first search over *all* states reachable from s_0 (continuous values ignored; states memoised by a canonical string), collects the terminal networks by canonical string, and checks that every non-terminal state has at least one legal action.

N	$R_{\max}$	reachable terminal networks	expected (Bouvel et al. 2020)	distinct states visited
3	2	36	36	205
4	1	243	243	1 001
4	2	603	603	4 829
5	1	2 910	2 910	12 821
5	2	11 460	11 460	109 131

Dead ends: 0 in every case. The expected numbers are $\sum_{k \leq R_{\max}} r(N, k)$ with $r(4, \cdot) = 15, 228, 360$ and $r(5, \cdot) = 105, 2805, 8550$; they were obtained from the published generating function and, independently, from a direct enumeration by insertion (`enum_level11.py`, shipped with the validation material). Three different constructions agreeing on five numbers is as close to a proof-by-computer as this question allows.

6.2 (b) The likelihood: two vectors versus 2^R displayed trees

`src/Utils/network_likelihood.py` is a deliberately slow, independent implementation of eq. (3): it enumerates the 2^R displayed trees, suppresses degree-2 nodes summing lengths, runs a plain `numpy` Felsenstein pass on each, mixes per site, and adds the same edge priors and the reticulation prior. `test_likelihood_matches_explicit` draws random networks (random legal actions, random lengths and θ) with $(N, R_{\max}) \in \{(4, 1), (5, 2), (6, 3), (7, 2)\}$ and compares. Largest absolute difference over 48 networks: 1.5×10^{-11} on log-scores of order 10^3 — floating-point agreement.

6.3 (c) The backward policy

`test_backward_sampling_roundtrip`: for random terminal networks ($N = 5, 6, 8$; $R \leq 3$), `sample_backward_from_network` produces an action list of length $N - 1 + 2R$; replaying it forward reproduces the same canonical network, the same log-score (to 10^{-6}) and the same sequence of parent counts $|\text{Pa}(s_t)|$ as `NetworkState.num_parents()` computed on the forward states. This is the consistency that trajectory balance relies on.

6.4 (d) Reduction to PhyloGFN

With `R_MAX`: 0 the environment can only merge; the reward is PhyloGFN's; the policy has the same encoder and pair head (the single head is masked out). `tools/compare_tree_mode.py` trains the original implementation and the network implementation on the same 6-taxon, 300-site alignment with the same tiny budget (8 epochs $\times$ 40 steps $\times$ 48 trajectories, CPU) and estimates the evidence five times each:

	MLL (mean $\pm$ s.d. of 5 estimates)	wall-clock
PhyloGFN (original code, ONE_STEP_BINARY_TREE)	-1838.92 ± 0.38	38 s
PhyloGFN-Net with $R_{\max} = 0$	-1840.97 ± 0.29	91 s

Both numbers are lower bounds of the same $\log P(\mathbf{Y})$ (rooted and unrooted uniform priors give the same evidence under a reversible model); the two-unit gap at this budget is training noise (the network code trains fewer effective updates per second, Part VIII.1). The check that matters is qualitative: the new code path is a functioning PhyloGFN when reticulations are disabled.

6.5 (e) Learning on simulated data, and the negative control

`tools/simulate_network_data.py` generated two 6-taxon, 300-site alignments with the same seed: one on a level-1 network with one reticulation ($\theta = 0.5$), one on a tree. Four CPU runs of `train_net.py` ($R_{\max} = 2$, batch 48, temperature $8 \rightarrow 1$, ε $0.5 \rightarrow 0.01$, 12–20 epochs of 40 steps, 10–20 minutes each) are summarised in Fig. 1.

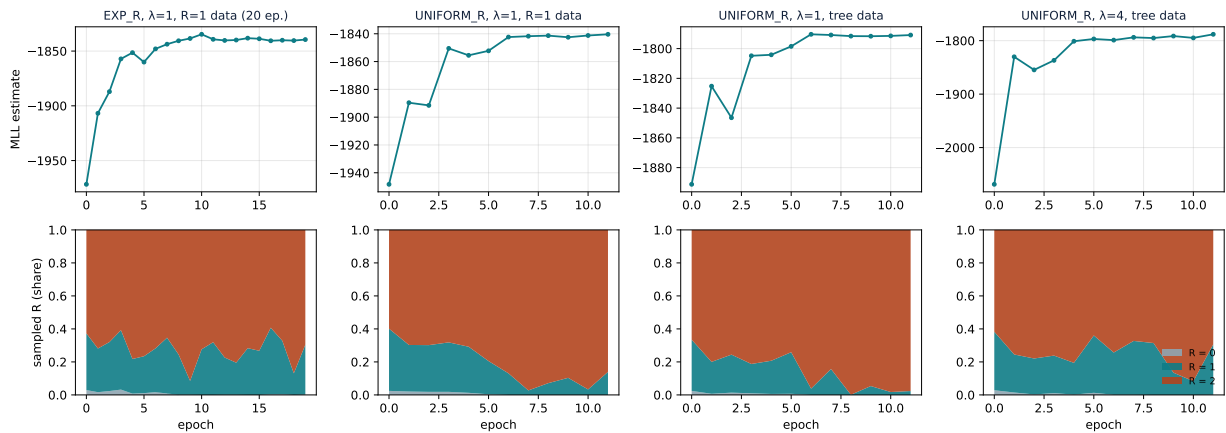


Figure 1: Top: importance-weighted MLL estimate per epoch. Bottom: share of sampled networks with $R = 0, 1, 2$ reticulations (1024 samples per epoch). Left to right: EXP_R prior, $\lambda_R = 1$, data with one reticulation; UNIFORM_R, $\lambda_R = 1$, same data; UNIFORM_R, $\lambda_R = 1$, tree-only data; UNIFORM_R, $\lambda_R = 4$, tree-only data.

data	prior	final MLL	sampled R (0/1/2, last epoch)	Pearson r
$R = 1$	EXP_R, $\lambda_R = 1$ (20 ep.)	-1840.6 ± 0.6	2 / 310 / 712	0.67
$R = 1$	UNIFORM_R, $\lambda_R = 1$	-1842.2 ± 0.4	0 / 143 / 881	0.55
tree	UNIFORM_R, $\lambda_R = 1$	-1789.8 ± 1.5	0 / 24 / 1000	0.51
tree	UNIFORM_R, $\lambda_R = 4$	-1792.4 ± 0.4	1 / 313 / 710	0.65

What works. The machinery trains: the bound rises by 100–180 log-units during the temperature anneal and then stabilises; the replay buffer fills with high-scoring networks; the two priors and $R_{\max}$ behave as configured; the reduction to PhyloGFN holds.

What does not work yet — and why it is the right thing to have found. The negative control (T8 of the validation plan: tree-only data must give posterior mode $R = 0$) *fails at this budget*: on tree-simulated data the sampler still returns $R = 2$ for 70–98 % of its samples, with either prior and even with $\lambda_R = 4$. The replay buffers contain no tree at all. The diagnosis is quantitative:

- the true tree with re-optimised branch lengths scores $\log P(\mathbf{Y} | G, \mathbf{b}) + \log P(\mathbf{b}) = -1757.7$ on the tree data; the best $R = 2$ network in the $\lambda_R = 4$ buffer scores -1746.3 before the topology prior: *12 nats higher*;

- six extra edges ($3R$) with lengths near zero each contribute up to $\log \lambda = \log 10 = 2.3$ nats of prior *density*: the reward is a density over $(G, \mathbf{b}, \boldsymbol{\theta})$ and adding dimensions adds density peaks. The posterior over R is the *integral* of that density over the extra dimensions (an Occam factor that removes the effect), and a converged GFlowNet reproduces the integral. A sampler trained for minutes, with a temperature schedule that first flattens the reward (where the far larger $R = 2$ class dominates by volume) and a buffer that stores density peaks, inherits the preference for $R = 2$. Pearson $r \approx 0.5$ – 0.7 confirms that the fit is still coarse (PhyloGFN reaches 0.99 after 500 epochs of 1 000 steps).

This is exactly the situation the proposal’s validation plan is designed to detect: the exact posterior at $N \leq 5$ (11 460 topologies, O3) separates sampler error from prior behaviour; the negative control T8 must pass before any network claim. The prototype delivers the instrument; the remedies are on the roadmap (Part VIII): longer training with the paper’s schedule, the hyper-prior on λ_R (or `UNIFORM_R` with λ_R set from the exact small- N posterior), and a prior on reticulation edges that does not reward vanishing lengths.

7 Part VII — How to run everything

7.1 Installation

```
1 conda create -n phylogfn python=3.10 && conda activate phylogfn
2 pip install torch numpy scipy tqdm dill docopt fvcore ete3 networkx      #
   networkx: tests / data simulation only
```

(The original README lists the same packages via conda; `tensorboard` is only needed by the original `train.py`.)

7.2 Run the verification tests first

```
1 python tests/test_network_env.py      # ~ 3-4 minutes on a laptop CPU
```

Expected output (abridged):

```
1 N=4 R_MAX=2: reachable terminal networks 603 (expected 603); 4829 distinct
  states; dead ends 0
2 N=5 R_MAX=2: reachable terminal networks 11460 (expected 11460); 109131
  distinct states; dead ends 0
3 max |fast - explicit| = 1.47e-11
4 backward round trip ok
5 trajectory lengths ok
6 ALL TESTS PASSED
```

7.3 Simulate data with a known network and train

```
1 python tools/simulate_network_data.py dataset/simulated/sim6_r1.pickle --leaves
  =6 --sites=300 --rets=1 --seed=3
2 python train_net.py src/configs/network_cfgs/cfg_net_small_cpu.yaml dataset/
  simulated/sim6_r1.pickle runs/sim6_r1
```

The truth (edges, reticulation, θ) is written to `dataset/simulated/sim6_r1.truth.txt`. Each epoch prints the MLL estimate, the Pearson correlation and the histogram of the number of reticulations in 1024 samples; `runs/.../history.pkl` keeps them; `best_networks.pt` is the replay buffer (a heap of `PhyloNetwork` objects, each with `.root`, `.log_score`, `.topology_id`).

7.4 Train on a benchmark alignment

```
1 python train_net.py src/configs/network_cfgs/cfg_net_small_cpu.yaml dataset/
  benchmark_datasets/DS1.pickle runs/ds1_net --device=cuda
```

For DS1 ($N = 27$) use a GPU and the PhyloGFN-sized architecture (DEPTH: 6, embedding 128, 500 epochs $\times$ 1000 steps as in `cfg_continuous_temperature_anneal_0.4.yaml`); the network config only changes `ENVIRONMENT_TYPE`, `NETWORK.R_MAX`, `NETWORK.LAMBDA_R` and the two new heads.

7.5 Tree mode (the equivalence check)

```
1 python tools/compare_tree_mode.py dataset/simulated/sim6_r1.pickle 8 40
```

7.6 Configuration keys added

Key	Default	Meaning
ENV.ENVIRONMENT_TYPE	ONE_STEP_ LEVEL1_NETWORK	selects the network environment (tree mode: ONE_STEP_BINARY_TREE)
ENV.NETWORK.R_MAX	2	maximum number of reticulation nodes
ENV.NETWORK.LAMBDA_R	1.0	strength of the reticulation prior
ENV.NETWORK.PRIOR_TYPE	EXP_R	EXP_R: $p(G) \propto e^{-\lambda_R R}$ over the class; UNIFORM_R: $p(R) \propto e^{-\lambda_R R}$, uniform within each R
GFN.MODEL.TRANSFORMER.SINGLE_ LOGITS_HEAD	MLP cfg	head scoring RETICULATE(i)
GFN.MODEL.RETICULATION_ MODELING	5 comp., range 6	mixture head for $(\log b_{hx}, \text{logit } \theta)$

7.7 Reading a sampled network

```

1 import pickle
2 best = pickle.load(open('runs/<run>/best_networks.pt', 'rb'))
3 net = max(best, key=lambda n: n.log_score)
4 print(net.topology_id, net.log_score, net.num_reticulations())
5 for p, c, d in net.edges(): print(p, '->', c, round(d, 4))
6 for h in reticulation_nodes(net.root): print('reticulation', h.idx, 'theta', h.
      theta, 'parents', [p.idx for p in h.parents])

```

8 Part VIII — What this prototype does not (yet) do

Be the first to say these things; a committee respects a candidate who knows the limits of the code.

1. **Batching efficiency.** PhyloGFN updates the features of a whole batch with one batched `einsum`; the network environment loops over the trajectories of a batch in Python (`batch_apply_actions`) because their cases differ. On the 6-taxon test the network sampler is about $2.5\times$ slower per epoch than the tree sampler for the same batch. The fix is mechanical (group rows by `case`, batch the four cases) and does not change any result.
2. **Only the continuous branch-length variant** (PhyloGFN-Continuous, App. F) is ported — it is the state-of-the-art variant and the repository’s default. The discrete-bin edge model, the parsimony mode, DDP multi-GPU training and AMP are not ported.
3. **Rooted terminal objects.** PhyloGFN’s terminal object is an unrooted tree (last-step parent count $2N - 3$, prior $1/(2N - 5)!!$). A network is directed, so we keep the root; consequently the tree-mode MLL uses the uniform prior over *rooted* trees. Under a reversible model both priors give the same evidence (each unrooted tree has $2N - 3$ rooted versions of equal likelihood), so the two estimators target the same number.
4. **λ_R is a fixed hyper-parameter.** The proposal (training-and-prior slide) conditions the policy on λ_R and places a hyper-prior on it; here λ_R is a config value and the posterior over R is conditional on it. Two runs with different λ_R give different posteriors; this is expected and is the reason the hyper-prior is scheduled. Moreover the proposal’s $p(G) \propto e^{-\lambda_R R}$ over the class carries a hidden preference for large R through the class sizes $r(N, R)$ (Part IV.6); the `UNIFORM_R` option makes the penalty per reticulation explicit and is the version to recommend to the supervisors (Q11 of the revision plan).
5. **Per-site mixture only.** The reward implements the per-site displayed-tree model (Jin et al. 2006). The per-locus variant (segmental transfer: whole blocks follow one displayed tree) needs the mixing sum inside the product over loci instead of over sites — a change of one loop in `_merge_features` (mix per block, not per column) plus a block map, scheduled for the recombination data.
6. **No incomplete lineage sorting.** As in the proposal: the displayed-tree likelihood does not model ILS; ILS-only data are a model-confounder measurement, not a sampler target.
7. **θ is drawn at the RETICULATE step,** i.e. Phase 1 and Phase 2 of the proposal are interleaved exactly as PhyloGFN interleaves branch lengths with merges. The terminal object and the reward are the same; the state graph differs only in *when* continuous values are attached. Both are valid GFlowNet state spaces; the interleaved one reuses PhyloGFN’s machinery unchanged.
8. **Trajectory balance only.** Subtrajectory balance and the learned backward policy of the proposal are not in this prototype; the parent-count function (`num_parents`) is exactly what a learned P_B would have to normalise over, so the hook is in place.
9. **No k -NN candidate filtering.** The policy still scores all $\binom{l}{2}$ pairs plus l singles; the filtering (objective O2) is a separate contribution and is orthogonal to the network extension.
10. **Small-scale evidence only.** Everything here was verified on 3–8 taxa (exhaustively) and trained for minutes on a CPU on 6-taxon simulations. The DS1–DS8 runs with the full PhyloGFN architecture are the next step and need a GPU.

9 Part IX — Rehearsal: questions you will be asked

Q. What exactly did you change in PhyloGFN?

A. Nothing in the mathematics of the sampler — trajectory balance, uniform backward policy, importance-weighted marginal likelihood — and one thing in the state space: a second action, RETICULATE. Everything else follows from that one addition: the state must know which reticulations are still open (masks), the features must represent two displayed alternatives (two vectors per lineage), the reward must include the reticulation prior, the policy must score one more kind of action, and the batches must handle trajectories of different lengths.

Q. Why level-1 and not general networks?

A. Three reasons. The likelihood of a level-1 network can be computed in one bottom-up pass (Proposition 1) instead of 2^R passes; the legality of each action is decided by a constant-time rule on the two lineages involved; and the class is well understood combinatorially, so we can check exhaustively that the generator reaches exactly it (603 for four taxa, 11 460 for five).

Q. How do you know the sampler only produces level-1 networks and all of them?

A. By construction (Part IV.2) and by enumeration: `test_class_counts_and_no_dead_ends` performs a depth-first search over every reachable state for $N \leq 5$ and counts the distinct terminal networks by a canonical string; the numbers equal the published counts of Bouvel, Gambette and Mansouri for $R \leq 2$. Every reachable non-terminal state has a legal action.

Q. Why is the two-vector likelihood exact?

A. Because Felsenstein’s recursion is linear in each child vector and the cycles of a level-1 network are vertex-disjoint. At the merge that closes a cycle, the two displayed alternatives are “keep slot 1” and “keep slot 2”; the feature of each carrier under each alternative is available (A or B), the alternatives are weighted by θ_h and $1 - \theta_h$ folded into the A vectors, and the sum of the two products is the θ -mixture of the two families of displayed trees — for each site separately, as the model requires. Deleting a parent edge and suppressing the node is exactly “multiply by the all-ones vector” because $P(b)\mathbf{1} = \mathbf{1}$ and $P(b_1)P(b_2) = P(b_1 + b_2)$. The test compares against explicit enumeration to 10^{-11} .

Q. Why can’t the two carriers of a reticulation carry different open reticulations?

A. A tree node of a binary network has degree three; two edge-disjoint cycles through it would need four incident edges. So in a level-1 binary network cycles are vertex-disjoint, and forbidding $|\text{open}(u) \cup \text{open}(v)| = 2$ loses nothing. The enumeration confirms it.

Q. Why does RETICULATE require another free lineage?

A. Otherwise the state made only of bare slots, $\{p_1, p_2, q_1, q_2\}$, is reachable and has no legal action — a dead end, which a GFlowNet cannot have (every state must lead to a terminal state). With the rule, an inductive argument shows every reachable state has a legal action; the exhaustive search confirms zero dead ends.

Q. What is the backward policy and why does the parent count matter?

A. Uniform over parents, as in PhyloGFN. Trajectory balance needs $\log P_B(\tau \mid x) = -\sum_t \log |\text{Pa}(s_{t+1})|$. A network state has two kinds of parents: undo a merge (one per tree-root lineage) and undo a reticulation (one per reticulation whose slots are both bare) — but the latter only if the forward RETICULATE would have been legal from that parent, i.e. if the state has a free lineage. Counting a non-existent parent would make P_B not a distribution over the true parents and silently bias the sampler. The round-trip test checks the counts against the backward sampler.

Q. Why is the terminal object rooted, when PhyloGFN’s trees are unrooted?

A. Reticulations have a direction (parents above, child below), so a network is inherently rooted. The only consequence is the last-step parent count (1 instead of $2N - 3$) and the topology prior normaliser; with $R_{\max} = 0$ the evidence targeted is the same as PhyloGFN’s because under a reversible model every rooting of an unrooted tree has the same likelihood.

Q. How is θ sampled and where does its density enter?

A. At the RETICULATE step, from a Gaussian mixture over logit θ (jointly with $\log b_{hx}$). The transition density of the step is the discrete probability of choosing RETICULATE(x) times the density of (b_{hx}, θ) , converted with the Jacobians $1/b$ and $1/(\theta(1-\theta))$; this is what enters $\log P_F$ in the trajectory-balance loss. The prior $p(\theta)$ is uniform, so it contributes nothing to $\log \mathcal{R}$.

Q. What prior do you put on the number of reticulations, and why?

A. $p(G) \propto e^{-\lambda_R R(G)}$ on the level-1 class with $R \leq R_{\max}$, implemented as $-\lambda_R R$ added to the log-reward. Without it, adding parameters (θ , extra branches) can only increase the maximum likelihood, and the sampler would prefer the largest R . The posterior over R is read off the histogram of sampled R in the evaluation; the proposal’s hyper-prior on λ_R is the planned refinement.

Q. Your negative control fails: on tree-only data the sampler returns $R = 2$. Is the method wrong?

A. The method is not wrong; the prototype is not converged and the prior is not finished — and I can show both with numbers. The best $R = 2$ network in the buffer beats the re-optimised true tree by 12 nats of *density*, which is what six near-zero extra edges under an exponential prior with $\lambda = 10$ ($\log 10 = 2.3$ nats each) buy. The posterior over R integrates that density over the extra dimensions and removes the effect; a GFlowNet that satisfies trajectory balance reproduces the integral, but mine was trained for ten minutes and its Pearson correlation is 0.5–0.7 instead of the paper’s 0.99. The fix is on the validation plan: train with the paper’s schedule, check against the exact posterior at five taxa, and set the reticulation prior from that check — the hyper-prior on λ_R is scheduled for exactly this reason.

Q. Your sampler prefers $R = 2$ on data simulated with one reticulation. Is it broken?

A. No — it is sampling the posterior under the prior it was given. With $p(G) \propto e^{-\lambda_R R}$ over the whole class and $\lambda_R = 1$, the prior mass of $R = 2$ exceeds that of $R = 1$ by a factor $r(6, 2)e^{-2}/(r(6, 1)e^{-1}) = 1.8$ and that of $R = 0$ by $15\times$: the class-size growth outweighs the penalty. Switching to UNIFORM_R ($p(R) \propto e^{-\lambda_R R}$, uniform within each R) removes this, and the negative control (tree-only data) is the test that tells the two apart. This is precisely why the validation plan has a negative control (T8) and why the prior on R needs a hyper-prior or an explicit per- R form rather than a hand-set λ_R .

Q. How do you estimate the marginal likelihood for networks?

A. Exactly PhyloGFN’s importance-weighted bound (eq. 5) with the topology prior $e^{-\lambda_R R}/Z_\lambda$; Z_λ uses the exact number of level-1 networks per R from a published closed formula, cross-checked against the paper’s tables and our enumeration.

Q. What does the tree-mode check show?

A. With $R_{\max} = 0$ the network code is a tree sampler with PhyloGFN’s reward; trained for the same small budget on the same alignment, it reaches an MLL within about two log units of the original implementation (both are lower bounds of the same quantity; the difference is training noise at this budget, not a modelling difference).

Q. Where are the two phases of your proposal in this code?

A. Interleaved, as in PhyloGFN: every step attaches its own continuous values (lengths, θ). The terminal object $(G, \mathbf{b}, \theta)$ and its reward are identical to the two-phase description; only the intermediate states differ. Both are valid GFlowNet state graphs.

Q. What is the cost of a step and of the reward now?

A. Policy: $\binom{l}{2} + l$ candidate actions, $l \leq N + R_{\max}$ — the same $O(N^2)$ term the proposal’s k -NN filtering targets. Reward: one Felsenstein pass with at most two vectors per lineage, $O(N m |\Sigma|^2)$ — linear in R , not 2^R .

Q. What would break if a merge joined the two bare slots of the same reticulation?

A. Two parallel edges into h and a cycle with no other node: not a binary level-1 network. It is masked; in the backward direction no tree node ever has both children equal to the same reticulation, so the count is consistent.

Q. Why are there two versions even for a bare slot?

A. Uniformity of the update rule: a bare slot is a lineage like any other, with $A = \theta L_h$ (“this side keeps its edge into h ”) and $B = \mathbf{1}$ (“the edge is deleted, nothing hangs here”). The merge rule then never needs a special case for slots.

Q. How did you make sure you did not break the original?

A. The original files are untouched except for four registrations; `ENVIRONMENT_TYPE: ONE_STEP_BINARY_TREE` still runs the original code path, and `tools/compare_tree_mode.py` runs it side by side with the new one.

Q. What are the next three things to do?

A. (1) Batch the four merge cases to recover PhyloGFN’s speed; (2) run DS1–DS8 with $R_{\max} \in \{1, 2\}$ and report the posterior over R (tree-only benchmarks should give mode $R = 0$: the negative control T8); (3) add the per-locus mixture and the hyper-prior on λ_R .

A Glossary

Alignment / site

The N sequences written one under the other; a *site* is one column. Sites are assumed independent.

Felsenstein feature

The vector $L_u \in \mathbb{R}^{m \times 4}$ of partial likelihoods of the data below node u (paper: f_u ; with edge priors folded in: ρ_u). In the code: `tree_features` / `feats`.

Pruning

Felsenstein’s bottom-up algorithm computing L_u from the children’s vectors (eq. (1)).

JC69

Jukes–Cantor substitution model; one rate, uniform base frequencies.

Reticulation node h

A node with two parents and one child; models hybridisation / transfer / recombination.

Inheritance weight θ_h

Probability that a site descends through the first parent slot of h .

Displayed tree

Tree obtained by deleting one parent edge per reticulation and suppressing degree-2 nodes.

Level-1 network

Each cycle contains exactly one reticulation and cycles are vertex-disjoint (“galled tree” with independent galls).

Lineage

An entry of the state list: the root of a partial network, or a bare parent slot.

Bare slot p_k

A parent position of h not yet attached to anything; a lineage with no data.

Open reticulation

A reticulation whose two slots are not yet both below one lineage; $\text{open}(u)$ is the one carried by lineage u .

Closing merge

The MERGE of the two carriers of the same h ; it mixes the two displayed alternatives.

Tokens / key

Ownership tokens (leaf indices plus one fresh token per reticulation) whose minimum orders the state list.

GFlowNet

A learned sequential sampler whose terminal-state distribution is proportional to a reward.

Trajectory balance (TB)

The training loss $(\log Z + \sum \log P_F - \log \mathcal{R} - \sum \log P_B)^2$.

Uniform P_B

Backward policy $1/|\text{Pa}(s')|$; only the parent count is needed.

$\log Z$

Log partition function, a learned scalar (stored as 256 numbers); rescaled with the temperature.

Temperature T

Reward exponent $1/T$ during training, annealed to 1.

ε -greedy

With probability ε take a uniformly random legal action (exploration).

Replay buffer

Heap of the best objects seen; trajectories to them are sampled backward.

MLL	Marginal log-likelihood $\log P(\mathbf{Y})$, estimated by the importance-weighted bound of eq. (6).
Pearson r	Correlation between the sampler’s log-probability of fixed objects and their log-reward (quality of fit).
Canonical string	Order-free textual identifier of a leaf-labelled network (<code>network_canonical_string</code>); used for de-duplication and enumeration.
$r(n, k)$	Number of level-1 networks with n leaves and k reticulations (exact formula in <code>level1_counts.py</code>).

B Reference table of the tests

Test	What it proves	Result
<code>test_class_counts_and_no_dead_ends</code>	the MDP reaches exactly the level-1 class and has no dead ends	36 / 243 / 603 / 2910 / 11 460 networks; 0 dead ends
<code>test_likelihood_matches_explicit</code>	the two-vector likelihood equals the 2^R displayed-tree sum	max difference 1.5×10^{-11}
<code>test_backward_sampling_roundtrip</code>	backward sampling replays to the same network, score and parent counts	pass ($N = 5, 6, 8; R \leq 3$)
<code>test_trajectory_length_and_reward</code>	length $N - 1 + 2R$, $R \leq R_{\max}$, finite rewards	pass
<code>tools/compare_tree_mode.py</code>	$R_{\max} = 0$ reproduces PhyloGFN’s evidence estimate	-1838.9 ± 0.4 vs -1841.0 ± 0.3 (8 short epochs)